ECEN/CSEN 123 Lab 8 Report

Tuesday 2:15 - 5:00

Austin Petersen, Dylan Thornburg, Madeline Follosco

I. Objectives

- Use the line-follower sensor to follow a straight or curved line.
- Determine how the geometry of your robot impacts this task.

III. Lab Procedure

Line Follower

Task 1:

Line Sensor Operation:

The QTRX-MD-08RC uses an R/C discharge circuit to create a digital timing signal that corresponds to the phototransistor measurement. Luckily, someone has already written the low level library to interface this device. *See* https://pololu.github.io/qtr-sensors arduino/class_q_t_r_sensors.html. This library can be used to read and calibrate the sensors. Make sure you understand the calibration procedure from the documentation.

Mount the QTRX-MD-08RC in accordance with your preparation and connect it to the Arduino. Suitable cables are in the lab. Don't power anything unless you understand what you are doing, or you will break stuff. If you need to drill, see the instructor. If your QTRX-MD-08RC does not have connector pins – see the instructor.

Set up part of a 10-wide LED display **with proper current-limiting resistors**. Connect these LEDs to the Arduino such that you can turn 8 adjacent LEDs on and off individually.
Using the QTR library – your program must do the following:

1. Print a prompt on the serial port monitor to calibrate for black. When you press a button or type in something (your choice), perform the black-level calibration. 2. Print a prompt on the serial port monitor to calibrate for white. When you press a button or type in something (your choice), perform the white -level calibration.
3. Enter a loop where you read the 8 sensors using the previous calibration. Compare each reading to a threshold value that determines whether you are seeing black or white. On the 8 LEDs (corresponding to the 8 sensors in order), light the LED's to show white detection and leave the LEDs dark for black detection.

Get black masking tape and white paper from the instructors and make a line. Place the line under the sensor. As you move it left and right, the dark band indication on the LEDs should move as well. (Demo #1)

Include your code and a photo of the system in operation in your lab report.

CODE

```
#include <QTRSensors.h>

int ledPin[8] = {2,3,4,5,6,7,8,9};
uint8_t sensorPin[8] = {22,23,24,25,26,27,28,29};
QTRSensors qtr;
uint16_t raw[8], black[8], white[8], thr[8];
int blackCalibrated = 0, whiteCalibrated = 0, thrCalibrated=0;

/*void readAvg(uint16_t out[8]) {
  uint32_t sum[8] = {0};
  for (int s = 0; s < 20; s++) {
    qtr.read(raw);
    for (int i = 0; i < 8; i++) sum[i] += raw[i];
    delay(5);
  }
  for (int i = 0; i < 8; i++) out[i] = sum[i]/20;
}*/

void setup() {
  Serial.begin(9600);
  Serial.println("Type b for BLACK, w for WHITE.");
  for (int i = 0; i < 8; i++) pinMode(ledPin[i], OUTPUT);
  qtr.setTypeRC();
  qtr.setSensorPins(sensorPin, 8); // expects unsigned 8 bit int
  qtr.setTimeout(2500); //us
}

void loop() {
  if (Serial.available()) {
    char c = Serial.read();
    if (c == 'b') {
      Serial.println("Calibrate for black.");
      delay(300);
      //readAvg(black);
      for(int i=0; i < 120; i++) qtr.calibrate();
      blackCalibrated = 1;
      //thrCalibrated = 0;
      Serial.println("Done.");
```

```
    }

    if (c == 'w') {
      Serial.println("Calibrate for white.");
      delay(300);
      //readAvg(white);
      for(int i=0; i < 120; i++) qtr.calibrate();
      whiteCalibrated = 1;
      //thrCalibrated = 0;
      Serial.println("Done.");
    }
  }

  if (!(blackCalibrated && whiteCalibrated)) return; //restart loop if
not calibrated

   /*if (!thrCalibrated) {
    for (int i = 0; i < 8; i++) thr[i] = (black[i] + white[i]) / 2;
    thrCalibrated = 1;
  } */

  qtr.readCalibrated(raw);
  //black is usually higher
  for (int i = 0; i < 8; i++) {
    digitalWrite(ledPin[i], (raw[i] > 750) ? LOW : HIGH);
    Serial.print("raw #" + String(i) + ": ");
    Serial.println(raw[i]);
    //Serial.print("threshold #" + String(i) + ": ");
    //Serial.println(thr[i]);
  }
  delay(30);

}
```

Task 2:

Do any construction, coding, or circuit design necessary for this task.

1. Mount the line-follower sensor and wheels in the permanent position and orientation that you plan to use for your final project.
2. Develop and test line-following code using the test patterns you developed. You should be able to follow a curve with any radius 200mm or larger and an angle of up to 45°. Initially test with the wheels elevated by moving the black line back and forth and seeing if you get the proper response, then test on the ground/bench.
3. Demo the line follower following at least one non-straight test pattern you developed. (Demo #2)
4. Measure the speed at which your line follower operates (meters/s or feet/s). Include in your report.

Based on the video, we calculate our speed (with a basePWM of 100) to be roughly a quarter of a meter/second (.25m/s).

5. Measure the maximum deviation from the line. Mark the center of your robot at the front with a pen and mount the paper ruler along the front edge. Use it to estimate the maximum deviation from the line. One good way to do this is to make a cell-phone video from right above, then view it frame by frame on your computer to find the maximum.

Based on the video, at most, the sensor deviates from the line by ~2cm but it always finds it again by pivoting.

6. Demo your line-following with no USB cable on the test courses I will supply. If your line follower works, it should follow my lines as well. (Demo #3)
7. The final project requires that you follow the actual line on the test field. You need to separately demo that for final project credit.

CODE for DEMO 2 & DEMO 3

```
#include <QTRSensors.h>

uint8_t sensorPin[8] = {22,23,24,25,26,27,28,29};
QTRSensors qtr;

uint16_t raw[8], black[8], white[8], thr[8];
int blackCalibrated = 0, whiteCalibrated = 0, thrCalibrated = 0;
```

```
#define PIN_EN_A 6 //change to whatever
#define PIN_IN1 32
#define PIN_IN2 34

#define PIN_EN_B 7 //change to whatever
#define PIN_IN3 33
#define PIN_IN4 35

#define LED_PIN 13
int basePWM = 35;
float Kp = 0.05f; //proportional || used for small changes and
normal line
float Kd = 0.00f; //derivative || used for sharp turns and corners
|| didnt end up needing
int lastError = 0;
int lastPos = 3500;


int clip(int x) { //never go below -255 or above 255
  if (x<-255) return -255;
  if (x>255) return 255;
  return x;
}

void setMotors(int leftPWM, int rightPWM) {
  leftPWM = clip(leftPWM);
  rightPWM = clip(rightPWM);

  //put back if too jittery
  //analogWrite(PIN_EN_A, 0);
  //analogWrite(PIN_EN_B, 0);

  if(leftPWM >= 0){
    digitalWrite(PIN_IN1, LOW);
    digitalWrite(PIN_IN2, HIGH);
  }
```

```
  else { // reverse
    digitalWrite(PIN_IN1, HIGH);
    digitalWrite(PIN_IN2, LOW);
    leftPWM = -1 * leftPWM; // PWM must be positive in analogWrite
  }

  if(rightPWM >= 0){
    digitalWrite(PIN_IN3, HIGH);
    digitalWrite(PIN_IN4, LOW);
  }
  else { // reverse
    digitalWrite(PIN_IN3, LOW);
    digitalWrite(PIN_IN4, HIGH);
    rightPWM = -1 * rightPWM; // PWM must be positive in
analogWrite
  }

  analogWrite(PIN_EN_A, leftPWM);
  analogWrite(PIN_EN_B, rightPWM);
}

int linePosBlack() {
  long numer = 0, denom = 0;
  for (int i=0; i<8; i++) {
    if (raw[i] > 750) {
      numer += (long)i * 1000; //0-7000
      denom += 1;
    }
  }
  if (denom == 0) return (lastPos < 3500) ? 0 : 7000;

  lastPos = (int) (numer/denom);
  return lastPos;
}

int startFlag;
void setup() {
  Serial.begin(9600);
```

```
  Serial.println("Type b for BLACK, w for WHITE.");

  qtr.setTypeRC();
  qtr.setSensorPins(sensorPin, 8);
  qtr.setTimeout(2500); //us

  pinMode(PIN_EN_A, OUTPUT);
  pinMode(PIN_IN1, OUTPUT);
  pinMode(PIN_IN2, OUTPUT);

  pinMode(PIN_EN_B, OUTPUT);
  pinMode(PIN_IN3, OUTPUT);
  pinMode(PIN_IN4, OUTPUT);

  pinMode(LED_PIN, OUTPUT);
  startFlag=1;
}

void loop() {

  if (startFlag) {
    digitalWrite(LED_PIN, HIGH);
    delay(5000);
    digitalWrite(LED_PIN, LOW);
    for(int i=0; i < 200; i++) qtr.calibrate();
    digitalWrite(LED_PIN, HIGH);
    delay(5000);
    digitalWrite(LED_PIN, LOW);
    startFlag=0;
  }

  qtr.readCalibrated(raw);
  int pos = linePosBlack();
  int error = pos - 3500; // 3500 = center

  int deriv = error - lastError;
  int correction = (int)(Kp * error + Kd * deriv);
  lastError = error;
```

```
  int leftPWM  = basePWM - correction; // if it steers the wrong
way, swap +/-
  int rightPWM = basePWM + correction;
  setMotors(leftPWM, rightPWM);
  delay(10);
}
```